

ACM

Agentic Context Management for Long Horizon Tasks

从“压缩上下文”到“Agent 自主管理短期 / 长期记忆”

Know-Why · Know-How · 架构 · 训练 · 实验 · 源码 · 生产化



Context Engineering evolution: compression → active editing → policy → lossless memory

Context Management 演进: ReAct → ReSum → LongSeeker → AdaCoM → ACM

研究基线: arXiv:2607.23809 v1 (2026-07-26) + 官方 GitHub 开源实现

整理日期: 2026-08-13

执行摘要

一句话结论

ACM 的关键不是“把摘要做得更好”，而是把 context management 变成 Agent 自己可调用的动作，同时把被压缩的原始消息无损保存到外部记忆中，再通过 query_memory 按需取回。它把 Working Context 从“完整历史”重新定义为“当前推理所需的高速缓存”。

- Know-why：长程 Agent 的真正瓶颈不是名义 Context Window 太小，而是有效上下文容量（effective context capacity）会被冗长工具输出、失败分支和重复搜索快速污染。
- 核心机制：只提供两个记忆工具——manage_context 与 query_memory。前者总结并 offload 原始历史，后者根据 summary_id 从原始历史中精确找回细节。
- 关键性质 1 - Lossless：摘要可以有损，但系统状态不能丢。原始消息进入 Long-Term Memory，因此压缩错误可恢复。
- 关键性质 2 - Agent-initiated：何时压缩由主 Agent 自己决定，不再依赖 90% context threshold 等外部硬规则。
- 训练创新：用“应当压缩 / 不应压缩”双约束 Teacher Annotation，专门学习 context-management timing；再用 teacher top-K 分布做 on-policy distillation / KD。
- 实验：Qwen3.5-9B 在 BrowseComp-Plus 从 ReAct 0.570 提升到 ACM-Post-Trained 0.727；DeepSearchQA 0.367 → 0.425；SWE-Bench Verified 0.489 → 0.530。
- 系统启发：下一代 Deep Research Runtime 应把 Working Context、Raw Trace / Evidence Store、Verifier 分开，Context Manager 只负责“投影”，而不应该成为唯一事实存储。

阅读地图

章节	重点问题
1-3	为什么 long context 仍会失败？ACM 与传统总结有什么结构差异？
4-6	manage_context / query_memory 如何构成 lossless memory loop？
7-9	Dual-constraint teacher pipeline、OPD/KD 到底在学什么？
10-12	实验、Pass@K、一条 83-turn trajectory 说明了什么？
13-15	源码如何组织？与 ReSum / LongSeeker / AdaCoM 如何统一理解？
16-18	生产化设计原则、Know-How 清单与后续研究方向。

1. 时代背景：为什么 1M Context 也解决不了 Long-Horizon Agent

Agent 的长程轨迹天然包含大量“低价值但高 token”的内容：搜索列表、网页原文、命令输出、失败尝试、重复验证以及模型自己的中间推理。即使底层模型支持超长上下文，这些内容仍会挤占注意力预算，并造成 context rot：关键证据被埋没、早期误判持续影响后续决策、模型开始重复已经做过的搜索。

因此要区分 nominal context window 与 effective context capacity。前者是最大 token 数，后者是模型在当前任务中仍能稳定利用的信息容量。ACM 的设计出发点正是：不要把 Context Window 当档案库，而应当把它当工作内存。

2. ReAct / Summary Agent 的结构性问题

ReAct 的历史是 append-only：每轮 action 与 observation 都永久留在上下文中。优点是信息不丢，但随着 horizon 增长，token 成本和噪声同时增长；一旦达到上限，Agent 被迫提前作答或终止。

Summary Agent 通过固定阈值压缩历史。它控制了长度，却引入两个新问题：第一，压缩何时发生由外部 heuristic 决定，与当前 reasoning phase 不一致；第二，原始消息被丢弃后，摘要中的遗漏无法恢复。ACM 同时针对“timing”和“recoverability”两点改造。

3. ACM 的核心抽象：短期 Working Memory + 长期 Raw Memory

ACM 将 in-context messages 视为短期工作记忆，把被移出的原始消息视为长期记忆。重要之处不在“模拟人类记忆”的比喻本身，而在于它形成了清晰的系统边界：短期层追求低噪声与当前相关性，长期层追求完整性与可恢复性。

于是 Context Management 不再等同于删除，而是从 active view 中“卸载 (offload)”信息。只要 raw trace 还存在，Working Context 就可以大胆地保持短小。

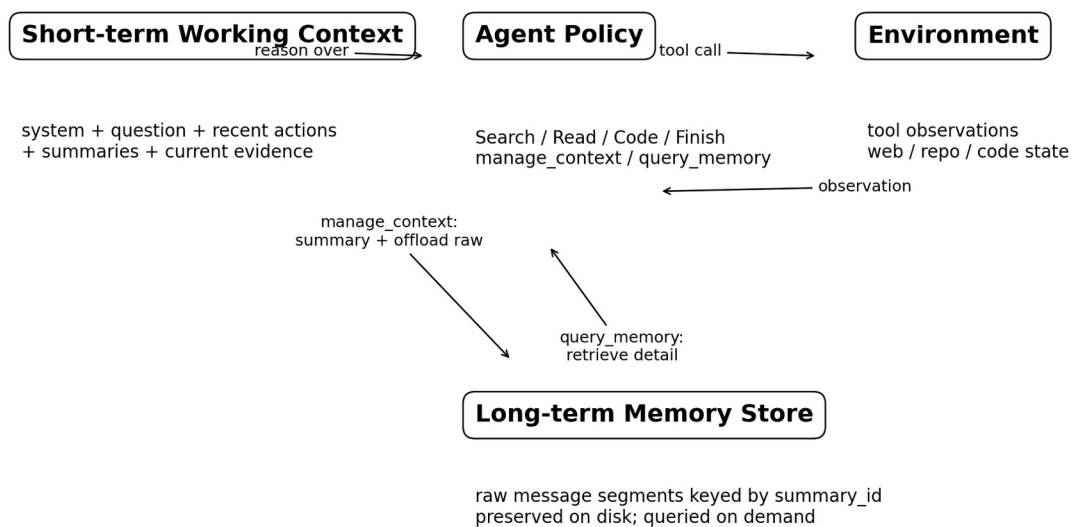


图2：ACM 的双层记忆架构——Working Context 是推理缓存，Long-Term Memory 保存可恢复的原始轨迹。

4. manage_context：压缩不是结束，而是一次状态迁移

当 Agent 主动调用 `manage_context` 时，系统会把“上一个 summary boundary 之后、当前调用之前”的消息压缩成一段摘要；同时把对应原始消息写入外部 workspace，并生成 `summary_id`。新的 Working Context 保留 system prompt、原问题、已有摘要和压缩后的新状态。

- 触发主体：Agent 自己。论文强调这是 ACM 与固定阈值 Summary Agent 的关键区别。
- 摘要目标：不是通用文本摘要，而是保存路径探索、关键证据、排除项、当前假设与未解决 gap。
- 存储语义：`summary_id` 是一个 handle，不只是引用标签；它是 Working Context → Raw Trace 的可逆映射入口。
- 工程含义：压缩动作最好是事务性的——summary 写入成功、raw segment 持久化成功、索引更新成功后再提交新的 active context。

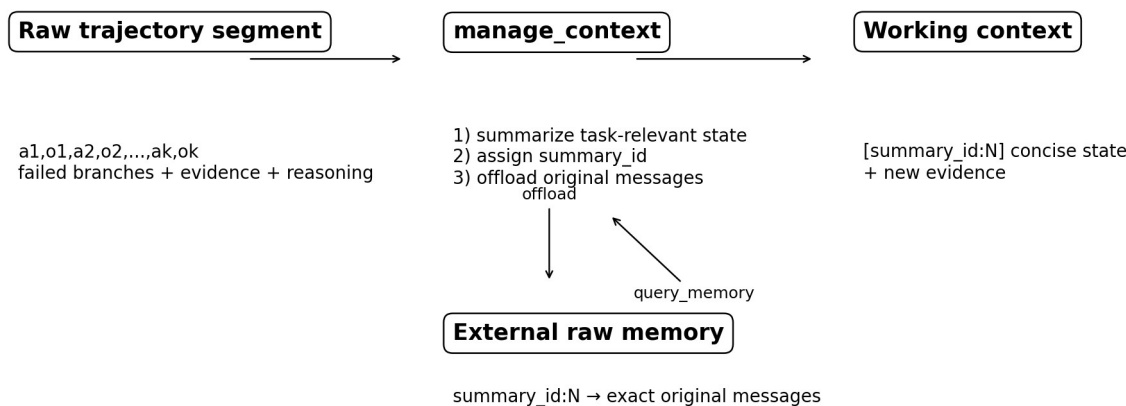


图3：Lossless compression 的关键：摘要允许有损，但被摘要掉的原始消息仍然可按 `summary_id` 恢复。

5. query_memory：把“记得过去”变成检索动作

`query_memory` 接收 `summary_id` 与具体查询，由 querier LLM 在该 summary 对应的原始消息上执行局部检索，再把相关细节作为 tool result 返回主 Agent。这使模型无需把所有历史恢复到 Working Context，只需要拿回当前问题需要的片段。

关键 know-how

External Memory 不应默认“整段 recall”。真正省 token 的模式是：summary 负责导航，`summary_id` 负责定位，`query` 负责选择性恢复。换言之，长期记忆的接口应该是 `Retrieve(query, scope)`，而不是 `LoadAll()`。

6. 为什么 ACM 可称为 lossless，而 ReSum 通常不是

维度	ReSum / 一般总结	ACM
历史压缩	把旧历史变成 summary	把旧历史变成 summary
原始内容	通常从 active episode 中丢弃	原始消息写入外部存储
恢复机制	依赖 summary 是否保真	query_memory 可回读 raw messages
触发	阈值 / interval 等固定规则	Agent 自主选择时机
风险	summary omission 可能不可逆	summary omission 可通过 memory query 修复

7. 真正难点不是“会压缩”，而是“什么时候压缩”

论文指出，即便 frontier model 也很难稳定掌握 context management timing。过晚压缩会遭遇 context rot 或 token limit；过早压缩则会破坏正在进行的局部推理，让模型不断丢失刚刚建立的细粒度线索。ACM 因此没有把训练目标写成“多调用 manage_context”，而是构造正反两类 timing supervision。

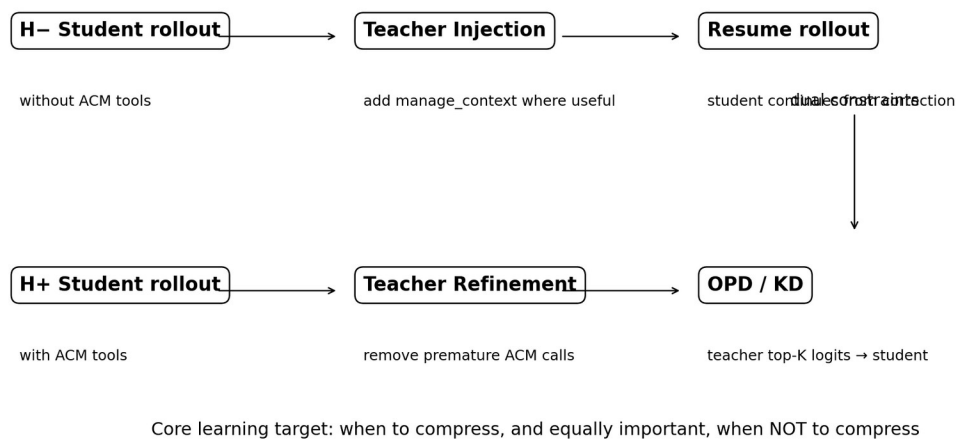


图4：Dual-constraint 数据构造——同时学习“哪里应该加ACM”与“哪里应该删除ACM”。

8. Dual-Constraint Teacher Annotation

训练数据从两类 student rollout 开始：H- 不提供 ACM tools，用来发现“如果当时压缩会更好”的位置；H+ 提供 ACM tools，用来发现学生“压缩得太早 / 太频繁”的位置。Teacher 依据参考答案审阅轨迹，但 annotation 不能泄露答案。

约束	Teacher 观察	修正动作	要学到的模式
Injection on H-	重复 query、进入 dead-end loop、上下文已足够大	插入 manage_context + 理由	识别 compression-worthy state
Refinement on H+	证据已足够却不作答、遗漏关键线索、压缩过早	替换成 search / open / finish	识别“不要压缩”的时机

Why it matters

Context action 的机会成本很高：一次错误压缩可能破坏当前 working set。因此训练必须包含 negative examples。只训练“何时调用 memory tool”会导致 over-management。

9. On-Policy Distillation / KD：为什么没有直接用重型 RL

论文采用 teacher-student distillation: teacher 对 student-generated trajectory 的 assistant token 给出 top-K next-token 分布 ($K=20$)，student 在 assistant token 上匹配 teacher distribution；system/user/tool-output token 被 mask。官方仓库进一步说明，受 GPU 资源限制，teacher scoring 与 KD training 被拆开，缓存 logprobs 后再训练，因此严格说是 off-policy；资源充足时可合并成真正 on-policy distillation。

- 优点：比端到端 RL 更容易稳定复现；context timing 可以通过局部 teacher correction 获得密集监督。
- 局限：teacher 的 context-management bias 会被蒸馏进学生；cached logprob 带来 rollout-policy 与 current-policy mismatch。
- 可迁移点：对高代价 Agent action，可先用 trajectory surgery + teacher correction 构造 demonstrations，再考虑 RL。

10. 主实验：ACM 不只是省 Token，也让 Agent 探索更深

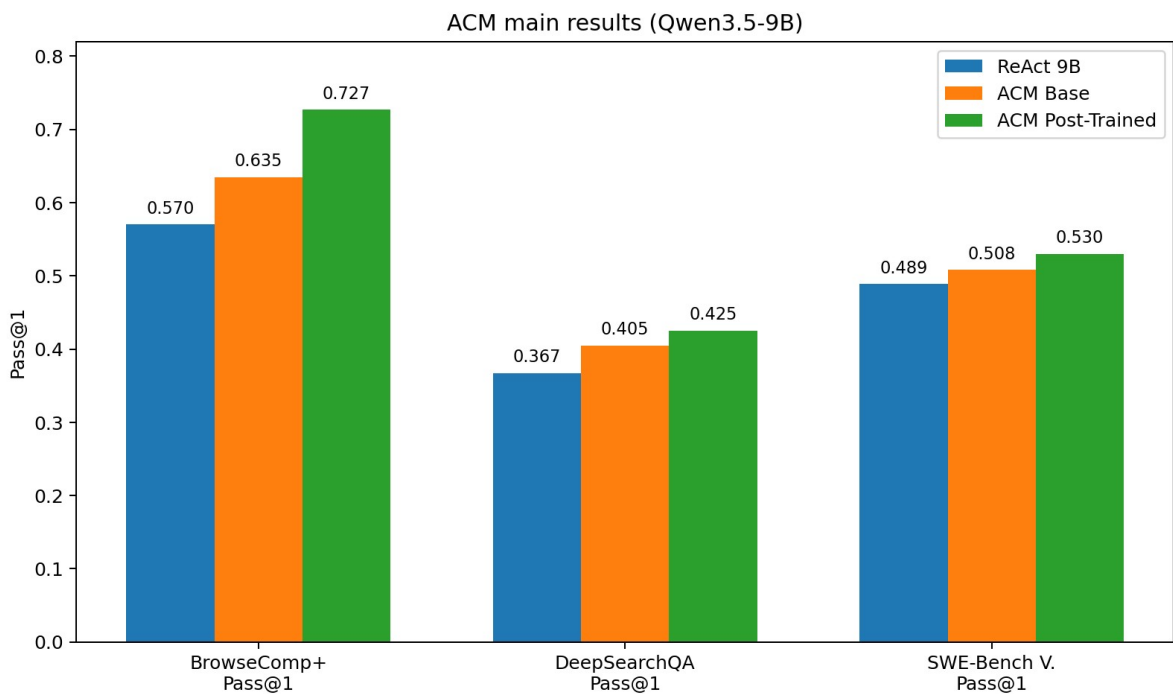


图5：Qwen3.5-9B 的Pass@1。ACM Base 已有增益，Post-Training 进一步提升。

方法	BrowseComp+	DeepSearchQA	SWE-Bench Verified
ReAct Qwen3.5-9B	0.570	0.367	0.489
ACM Base	0.635	0.405	0.508
ACM Post-Trained	0.727	0.425	0.530

相对 ReAct，论文总结 Qwen3.5-9B 在 BrowseComp-Plus、DeepSearchQA、SWE-Bench Verified 分别约提升 27%、16%、8%（相对百分比）。更重要的是，ACM-Post-Trained 的工具调用数往往更多：它不是通过“少做事”换取更短 context，而是通过不断清理 working memory，让 Agent 有预算继续搜索。

11. Peak Token：压缩的目标不是平均长度，而是削峰

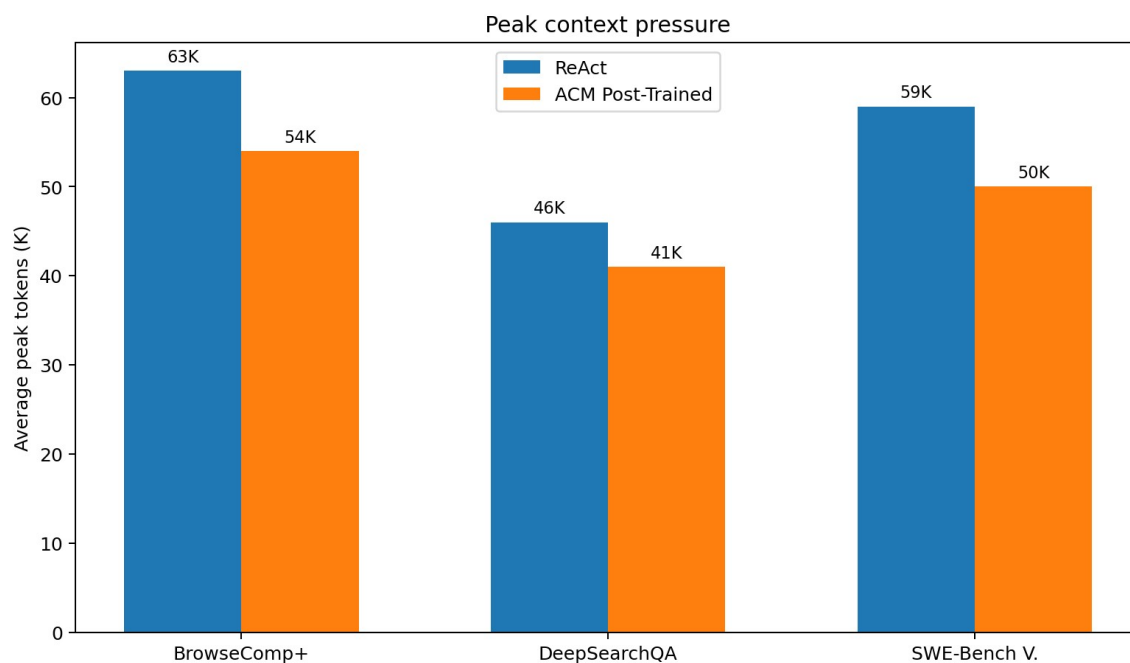


图6：ACM Post-Trained 相比 ReAct 降低三项任务的平均峰值上下文压力。

对于长程 Agent，peak token 比 average token 更关键。平均长度看起来安全，并不代表某个关键阶段不会突然被大网页、日志或代码输出推过 Context Limit。ACM 允许 Agent 在“预计下一阶段会继续膨胀”时提前 `manage_context`，因此能降低峰值并延长 horizon。

12. Pass@K：Context Management 改善的是稳定性

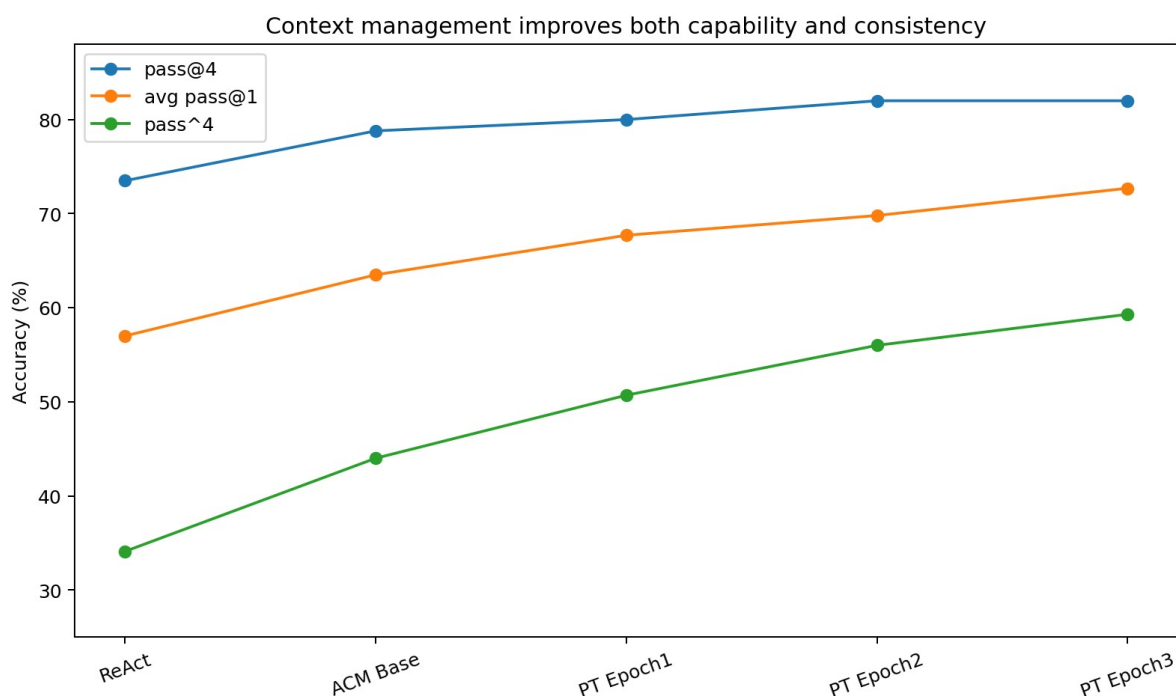


图7: BrowseComp-Plus 上, 随着ACM post-training, $\text{pass}@1$ 与 4 次全部成功的 pass^4 都持续提高。

论文用 $\text{pass}@4$ 表示能力边界, 用 pass^4 表示多次独立 trial 的一致成功。ReAct 的两者差距很大, 说明模型“偶尔能解出”但长程轨迹非常不稳定; ACM 训练后 pass^4 从 34.1% 提高到 59.3%, 表明 context control 的价值之一是降低 trajectory variance。

13. 一条 83-turn Trajectory：ACM 如何把 222K raw history 装进 128K 模型

论文展示一个 BrowseComp-Plus 5-constraint multi-hop 案例：ACM-Post-Trained 在 83 turns 中使用 63 次 search、9 次 get_document、7 次 manage_context、5 次 query_memory。若不压缩，raw history 会增长到约 222K；实际 active context 峰值约 98K。没有压缩时轨迹在 turn 47 左右就会越过 128K base limit。

- 1. Turn 10：模型注意到搜索开始围绕相似 clue set 循环，主动 manage_context。
- 2. Turn 18：再次压缩，为后续多跳搜索释放空间。
- 3. Turn 49：query_memory 找回旧摘要中的候选线索，但模型没有直接相信，而是继续验证。
- 4. Turn 58：active context 接近高水位，再次把大段历史压成仅约 2.3K 的摘要状态。
- 5. Turn 67-79：继续找到 dissertation / paper 等关键证据，并再做一次 manage_context。
- 6. Turn 82：Working Context 仅约 20K，证据已经闭合，Agent 最终作答。

最有价值的观察
manage_context 不是“快到上限才救火”。训练后的 Agent 会根据搜索阶段、重复模式和下一步工作量提前做 context scheduling。它本质上是资源规划动作。

14. 官方开源实现：从论文概念到 Runtime

官方仓库已公开数据、student rollouts、teacher top-K logprob cache、三轮 OPD checkpoint，以及完整数据生成与训练 pipeline。README 给出的主链路是：Student Rollout → Teacher Annotate → Student Resume → Teacher Label → KD Train → Evaluation。

目录 / 文件	职责
src/runner.py / src/run.py	Agent loop、工具执行、rollout 驱动
src/history.py	history / context state 管理
src/prompts.py	基线与 memory-tool prompt
src/teacher_guide/	Teacher annotation + student resume
distill/	teacher logprob scoring、KD loss、trainer
train/	trajectory preprocessing、checkpoint merge
scripts/	pipeline config、per-stage launcher
data/	BrowseComp-Plus train/eval question set

- Student：Qwen3.5-9B。

- Teacher / OPD scorer: Qwen3.5-397B-A17B; Teacher annotation 阶段 README 还使用 GPT-5。
- Retrieval: 默认本地 BM25 (Pyserini) ; 也支持 Qwen3-Embedding-8B dense retriever。
- 仓库将大规模 rollout 与 checkpoint 放在 Hugging Face collection, 代码仓库保持较轻。

15. ACM vs ReSum vs LongSeeker vs AdaCoM

Context-management design space

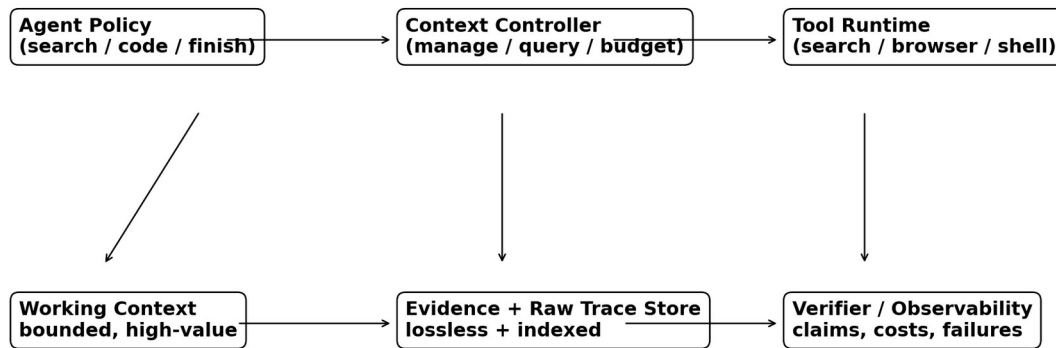
System	Who triggers?	Raw history preserved?	Learned?	Core abstraction
ReSum	external threshold / interval	No	Yes	restartable summary
LongSeeker	main agent	Partly / edited	SFT	context meta-actions
AdaCoM	external manager	implementation-dependent	RL	agent-compatible context policy
ACM	main agent	Yes	OPD/KD	lossless short+long-term memory

图8：四条 Context Management 路线的核心差异。

这四类方法不是简单的“谁更先进”，而是控制权和持久化语义不同。ReSum 把 context compression 做成固定策略；LongSeeker 把编辑动作纳入主 Agent action space；AdaCoM 把 context policy 外置以兼容冻结 / 闭源 Agent；ACM 则强调“主 Agent 自主触发 + 原始信息无损外存 + 可按需查询”。

场景	更适合的思路	理由
不能训练主 Agent / 闭源 API	AdaCoM	外部 Manager 可单独训练和路由
需要细粒度编辑当前 history	LongSeeker	Skip/Compress/Rollback/Snippet/Delete action 更丰富
快速给 ReAct 增加长程能力	ReSum	改动小、容易 plug-in
需要可恢复、审计、长程检索 / coding	ACM	raw trace 不丢，summary 可以作为可逆 view

16. Production Upgrade : 从 ACM 论文到真正的 Agent Runtime



Production principle: active context should be lossy; system state should not be.

图9：生产级 Context Runtime 建议：Working Context、Lossless Store 与 Verifier 分离。

- Working Context = View：只能存当前决策需要的摘要、证据 handle、任务约束和近期状态。
- Raw Trace Store = Source of Truth：原始 tool observation、网页、文件 diff、日志必须独立持久化，不能被摘要替代。
- Evidence Store = Structured Memory：对 research task 建议把 claim / source / excerpt / timestamp / confidence 从 raw trace 中结构化出来。
- Context Controller = Scheduler：触发 manage 的信号不只 token ratio，还包括重复查询率、information gain、branch completion、pending subgoal 数量。
- Retrieval should be scoped：query_memory 至少支持 summary_id / time range / tool type / source / semantic query 等 scope。
- Verifier before eviction：若当前 context 中存在未持久化的关键事实、用户约束或环境状态，禁止压缩提交。

17. 可累积的 Know-Why / Know-How

原则	Know-Why	Know-How
Context $\neq$ Memory	把所有历史放进 prompt 会让推理层承担存储职责	Working Context 只做 active view；原始状态独立持久化
Lossy View + Lossless State	摘要不可避免遗漏，但遗漏不该永久损坏任务	summary + raw handle；需要时 query_memory
Timing 是 Policy	固定 90% 阈值与 reasoning phase 无关	让 Agent / Manager 根据循环、阶段、预算主动触发
Negative supervision	只教“何时压缩”会导致过度压缩	同时构造“这里不该压缩，应继续 search/read/finish”的轨迹
Peak budget 优先	长任务常因单次大 observation 突然爆窗	监控 predicted next-step growth 与 peak tokens
Memory query 要局部化	整段恢复会重新制造 context rot	query + scope + summary_id，返回 task-relevant excerpts
Consistency 是一等指标	Agent 成功一次不代表稳定	同时评估 pass@K 与 all-K success / variance

统一公式

Reliable Long-Horizon Agent $\approx$ Task Policy $\times$ Context Policy $\times$ Lossless State Store $\times$ Retrieval Policy $\times$ Budget Controller $\times$ Verifier

18. 如果今天实现一个 ACM-style Research Agent

- 先把 raw tool outputs 从 messages 中抽离到 object store / filesystem，给每个 observation 一个稳定 handle。
- 实现 manage_context：生成结构化 summary（facts / explored / rejected / open gaps / next actions），并原子持久化被折叠的 raw segment。
- 实现 query_memory(query, scope)：先检索 metadata，再对命中 raw segments 做局部 rerank / extraction。
- 在 prompt 中明确 Working Context 是短期记忆、Long-Term Store 可按需查询，并给 Agent token / step budget telemetry。
- 先用 heuristic timing 建 baseline，再收集 failure trajectories，让 stronger teacher 标注“应压缩 / 不应压缩”的 turn。
- 做 SFT / distillation 后重点看：peak tokens、tool horizon、重复搜索率、pass^K，而不仅是单次 accuracy。
- 最后才考虑把更多 context actions（rollback / snippet / delete）加入 action space；动作越丰富，credit assignment 越难。

从系统设计角度，ACM 最值得迁移的不是两个工具名，而是一个原则：Active Context 可以被大胆重写，但事实状态必须能被恢复和审计。只要这个边界成立，Context Engineering 就从一次性的 Prompt Compression 变成真正的 Runtime Resource Management。

主要来源

[S1] Xiaochuan Li et al. ACM: Agentic Context Management for Long Horizon Tasks. arXiv:2607.23809, 2026-07-26.

[S2] Official GitHub: lixiaochuan2020/agentic-context-management. README, source tree, released rollouts / checkpoints.

[S3] ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization. arXiv:2509.13313.

[S4] LongSeeker: Elastic Context Orchestration for Long-Horizon Search Agents. arXiv:2605.05191.

[S5] Learning Agent-Compatible Context Management for Long-Horizon Tasks (AdaCoM). arXiv:2605.30785.

说明：本报告中的 ACM 机制、训练流程与数值以论文 / 官方仓库为依据；“生产化升级”和统一架构公式属于基于这些结果的工程推演，不是论文原文结论。